

Informe grupal

Middleware PSR-15 y endurecimiento de seguridad

Desarrollo de Aplicaciones Web II

Integrantes: _____

Docente: _____

Unidad: II - Semana 6

Proyecto: Sistema MVC PHP con autenticación y RBAC

Fecha: Junio de 2026

1. Objetivo

Implementar una tubería de middlewares inspirada en PSR-15 para centralizar autenticación, validación de fingerprint, autorización RBAC y cabeceras de seguridad. La solución evita que una petición no autorizada llegue al controlador y emplea contratos fuertemente tipados.

2. Arquitectura implementada

La petición sigue el siguiente flujo:

```
Petición HTTP
-> ServerRequest
-> MiddlewareDispatcher
    -> AuthenticationMiddleware
    -> AuthorizationRBACMiddleware
-> ControllerRequestHandler
-> Controlador MVC
-> Response HTTP
```

Los componentes se encuentran en `Libraries/Core/Http`, `App/Http/Middleware` y `App/Repositories`. La integración general se realiza desde `index.php`.

3. Lista de cotejo validada

N.º	Criterio técnico	Estado	Evidencia
1	Interfaces <code>MiddlewareInterface</code> y <code>RequestHandlerInterface</code> .	Cumple (2)	<code>Libraries/Core/Http/Interfaces.php</code>
2	Pipeline compuesto con <code>array_reduce</code> .	Cumple (2)	<code>MiddlewareDispatcher.php</code> usa <code>array_reduce</code> y <code>array_reverse</code> .
3	Autenticación mediante sesión y fingerprint.	Cumple (2)	<code>AuthenticationMiddleware.php</code>
4	Regeneración del identificador de sesión.	Cumple (2)	<code>NativeSession.php</code> ejecuta <code>session_regenerate_id(true)</code> .
5	RBAC mediante ORM o repositorio.	Cumple (2)	<code>PermissionRepository.php</code> y <code>LoginModel.php</code> .
6	Cabecera <code>Content-Security-Policy</code> .	Cumple (2)	Configurada globalmente en <code>index.php</code> .
7	Cookie de sesión con <code>HttpOnly</code> .	Cumple (2)	<code>session_set_cookie_params</code> antes de <code>session_start</code> .

N.º	Criterio técnico	Estado	Evidencia
8	Cortocircuito antes del controlador.	Cumple (2)	RBAC devuelve respuesta 403 sin ejecutar el siguiente handler.
9	Tipado estricto y tratamiento inmutable de la petición.	Cumple (2)	<code>declare(strict_types=1)</code> , request inmutable y sesión encapsulada.
10	Respuestas fundamentadas al cuestionario.	Cumple (2)	Incluidas en la sección 5 del presente informe.

Resultado estimado: 20/20 puntos.

4. Evidencias de código

4.1 Contratos de interfaz

```
interface RequestHandlerInterface
{
    public function handle(ServerRequestInterface $request): ResponseInterface;
}

interface MiddlewareInterface
{
    public function process(
        ServerRequestInterface $request,
        RequestHandlerInterface $handler
    ): ResponseInterface;
}
```

4.2 Composición funcional del pipeline

```
$pipeline = array_reduce(
    array_reverse($this->middlewares),
    function (callable $nextStack, MiddlewareInterface $middleware): callable {
        return function (ServerRequestInterface $req)
            use ($nextStack, $middleware): ResponseInterface {
                // El wrapper representa al siguiente handler.
                return $middleware->process($req, $wrapper);
            };
    },
    fn(ServerRequestInterface $req): ResponseInterface
    => $this->fallbackHandler->handle($req)
);
```

4.3 Autenticación y fingerprint

```
if (!$this->session->has('user_id')) {
    return Response::redirect($this->loginUrl());
}

$fingerprint = md5(
    (string) $request->getAttribute('user_agent', '') .
    (string) $request->getAttribute('remote_addr', '')
);

if ($this->session->get('fingerprint') !== $fingerprint) {
    $this->session->destroy();
    return Response::redirect($this->loginUrl() . '?error=session_invalid');
}
```

4.4 Regeneración y cookie segura

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'secure' => !empty($_SERVER['HTTPS']),
```

```

        'httponly' => true,
        'samesite' => 'Lax',
    ]);

    public function regenerate(): void
    {
        session_regenerate_id(true);
    }

```

4.5 Autorización RBAC y cortocircuito

```

$tieneAcceso = $this->repositorio->checkUserPermission(
    $userId,
    $this->permisoRequerido
);

if (!$tieneAcceso) {
    return new Response(
        'Error 403: No tienes permiso para: ' . $this->permisoRequerido,
        403
    );
}

return $handler->handle($request);

```

4.6 Cabeceras de seguridad

```

header("Content-Security-Policy: default-src 'self'; script-src 'self'");
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: DENY");

```

5. Cuestionario técnico

Pregunta de evidencia 1

¿Cuál es la principal ventaja de utilizar interfaces fuertemente tipadas?

Las interfaces establecen un contrato verificable: el middleware sabe que el handler posee el método `handle()`, recibe un `ServerRequestInterface` y devuelve un `ResponseInterface`. Esto permite detectar incompatibilidades tempranamente, facilita el autocompletado y evita la ambigüedad de un callable genérico.

Pregunta de evidencia 2

¿Por qué se utiliza `array_reverse()` antes de `array_reduce()`?

La reducción construye la tubería desde la función final hacia afuera. Al invertir la colección, el primer middleware registrado queda como capa exterior: se ejecuta primero al entrar y último al regresar la respuesta. Así se conserva el orden esperado de la estructura de cebolla.

Pregunta de evidencia 3

¿Cómo mitiga `session_regenerate_id(true)` la fijación de sesión?

Después de autenticar al usuario se genera un identificador nuevo y se elimina el anterior. Si un atacante había impuesto o conocido previamente el ID de sesión de la víctima, ese identificador deja de representar la sesión autenticada y ya no puede reutilizarse para ingresar.

Pregunta de evidencia 4

¿Qué significa el cortocircuito y cómo protege al controlador?

Significa devolver una respuesta desde el middleware sin llamar a `$handler->handle($request)`. Cuando RBAC deniega el permiso, retorna HTTP 403; por ello el controlador no se instancia ni ejecuta acciones de consulta, modificación o eliminación inseguras.

Pregunta de evidencia 5

¿Qué función cumple `nosniff`?

`X-Content-Type-Options: nosniff` ordena al navegador respetar el tipo MIME declarado por el servidor. Evita que archivos servidos como texto, imágenes u otros tipos sean reinterpretados como JavaScript o CSS, reduciendo ataques XSS basados en confusión de MIME.

6. Pruebas realizadas

Prueba	Resultado
Validación de sintaxis PHP	Todos los archivos analizados sin errores.
Acceso al login	HTTP 200.
Acceso anónimo a una ruta protegida	HTTP 302 hacia Login; controlador no ejecutado.
Permiso RBAC denegado	HTTP 403 y handler final no invocado.
Cookie PHPSESSID	Incluye HttpOnly y SameSite=Lax.
Cabeceras HTTP	CSP, nosniff y X-Frame-Options presentes.

7. Conclusiones

La seguridad transversal quedó centralizada en una tubería tipada. La autenticación se ejecuta antes que RBAC, el fingerprint vincula la sesión al entorno del usuario y las respuestas de rechazo impiden alcanzar el controlador. La petición se maneja de forma inmutable y el acceso a la sesión nativa se encuentra encapsulado mediante `SessionInterface`.

Archivo generado como parte del entregable académico. Los nombres de integrantes y docente deben completarse antes de la presentación final.